



Amazon Relational Database Service (RDS)

Lecture Slides

These lecture slides are provided for personal and non-commercial use only

Please do not redistribute or upload these lecture slides elsewhere.

Good luck on your exam!

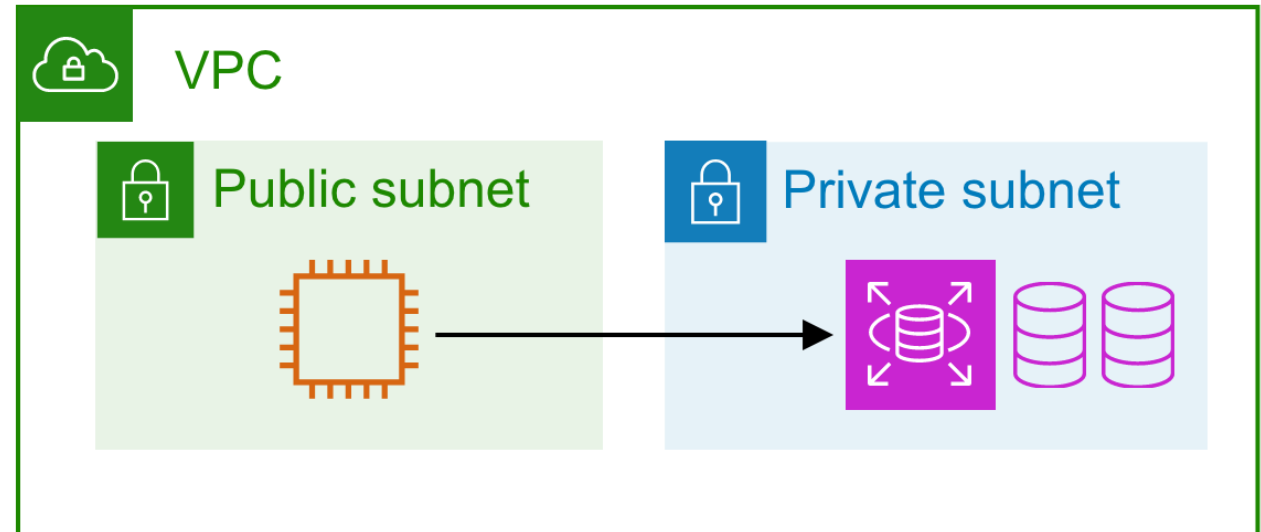
Introduction to RDS



Relational Database Service (RDS) is a managed service for **multiple** open-source and proprietary relational databases.

Features:

- Supports different kinds of db engines
- Automatic and manual backups
- Multi-AZ
- Read Replicas
- Performance Insights
- Customize DB parameters
- RDS Proxy for a connection pooler
- Various methods of authentication
- Blue/green deployments
- and more....



Introduction to RDS



MySQL

- The most popular open-source SQL database that was purchased and is now owned by Oracle.
- Offers replication and partitioning features for scalability and availability.



MariaDB

- Oracle bought MySQL, MariaDB made a fork (copy) of MySQL under a different open-source license.
- Continues to maintain high compatibility with MySQL, ensuring a drop-in replacement capability.



Postgres (PSQL)

- Most popular open-source SQL database among devs. Has rich-features over MySQL but at added complexity.
- Supports advanced data types and functions, such as JSON, XML, and key-value pairs for app development.



Oracle

- Oracle's proprietary SQL database. Well used by Enterprise companies. You have to buy a license to use it.
- Features a complex architecture that supports large-scale databases and multi-tiered applications.



Microsoft SQL Server

- Microsoft's proprietary SQL database. You have to buy a license to use it.
- Integrates seamlessly with other Microsoft products and services, including Azure cloud services.



IBM DB2

- IBM's proprietary SQL database. You have to buy a license to use it.
- Known for its high performance and scalability in large enterprise environments.



Aurora

- Fully managed database service that's compatible with MySQL and PostgreSQL.
- Automatically divides your database volume into 10GB segments spread across many disks, enhancing performance and reliability.

RDS - Encryption

- You can turn on **encryption-at-rest** for all RDS engines
- You may not be able to turn encryption on for older versions of some engines.
- It will also encrypt the automated backups, snapshots, and read replicas.
- Encryption is handled using the AWS Key Management Service (KMS)
- You can only turn encryption on during creation
 - You can take snapshots and launch new instances with encryption turned on



Encryption

☒ **Enable Encryption**
Choose to encrypt the given instance. Master key ids and aliases appear in the list after they have been created using the Key Management Service(KMS) console. [Info](#)

Master key [Info](#)

(default) aws/rds ▼

Encryption-in-Transit is provided by default via the databases DNS Endpoint

RDS – Backup

AWS RDS provides backup options in two main categories: Automated Backups and Manual Snapshots.

Automated Backups

- **Retention Period:** Choose a period between **0 and 35 days**.
 - Setting the retention period to **0 days** disables automated backups.
- **Point-in-Time Recovery (PITR):**
 - Allows restoration with **1-second precision** within the retention period (up to 35 days).
 - AWS replays the transaction logs from a full backup to the specified point in time.
- **Transaction Logs:** Continuously stored throughout the day.
- **Storage Location:** All backups are stored in Amazon S3.
- **Backup Window:** You can define a preferred backup window.
- **I/O Suspension:** Storage I/O may be paused briefly during backup.
- **No Additional Charge:** Automated backups are included in the RDS pricing.

```
aws rds modify-db-instance \  
--db-instance-identifier my-sample-db \  
--backup-retention-period 7 \  
--preferred-backup-window 03:00-04:00 \  
--apply-immediately
```

Manual Snapshots

- **User-Triggered:** Taken manually by the user.
- **Persistence:** Snapshots persist even if the RDS instance is deleted.
- **Copy Across Regions:** Snapshots can be copied to other AWS regions.
- **Sharing:** Snapshots can be shared with other AWS accounts.
- **Export:** Manual snapshots can be exported to Amazon S3 as a data file.
- **Additional Storage Cost:** There is an extra charge for the storage of manual snapshots.

```
aws rds create-db-snapshot \  
--db-snapshot-identifier backup-before-big-deploy \  
--db-instance-identifier my-db-instance
```

The database **must be in the "available" state** to create a backup

RDS – Restoring Backup

Restoring a Backup for automated and manual **creates a new RDS instance** and restores the data to that instance.

Restoring a manual snapshot via the AWS CLI

```
aws rds restore-db-instance-from-db-snapshot \  
--db-instance-identifier restored-db-instance \  
--db-snapshot-identifier my-db-snapshot
```

Restoring a Point-in-Time (PITR) Backup via Automatic Backups via the AWS CLI

```
aws rds restore-db-instance-to-point-in-time \  
--source-db-instance-identifier source-db-instance \  
--target-db-instance-identifier restored-db-instance \  
--restore-time "2023-03-29T15:45:00Z"
```

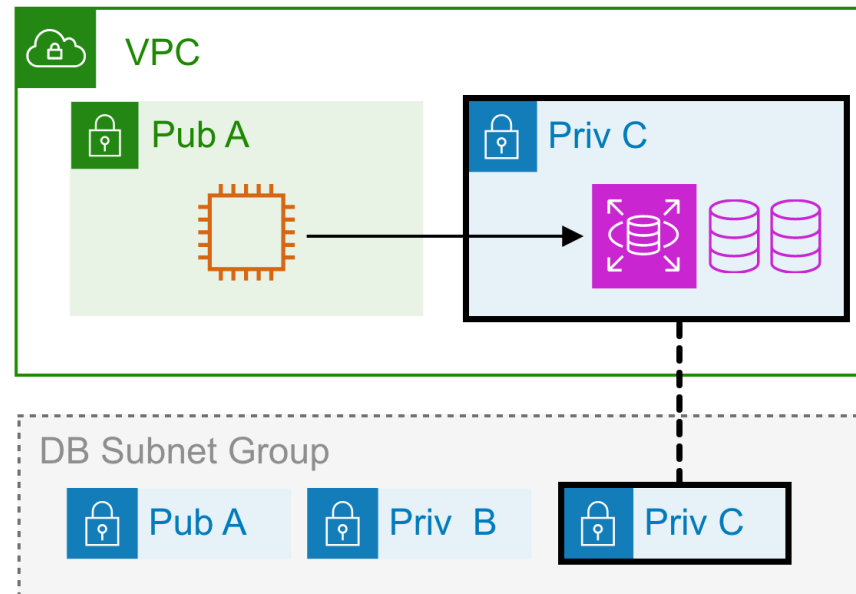


Restoring backups is not a fast process because it creates a new database, consider that for Recovery Time Objectives (RTO)s

RDS – Subnet Groups

DB subnet group is a **collection of subnets** (usually private subnets) that you create in a VPC and that you then designate for your DB instances.

- Each DB subnet group should have subnets in at least two Availability Zones in a given AWS Region.
- RDS will choose a subnet from your subnet group to deploy your RDS Instance
- Subnets in a DB subnet group are either public or private
- For a DB instance to be publicly accessible, all of the subnets in its DB subnet group must be public



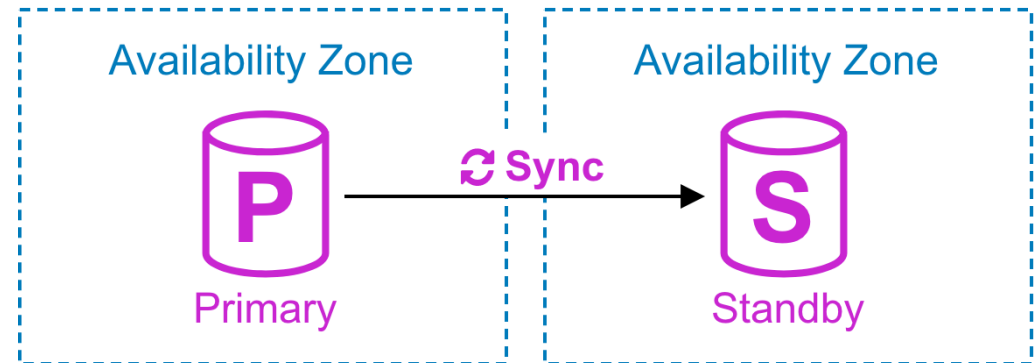
RDS - Multi AZ



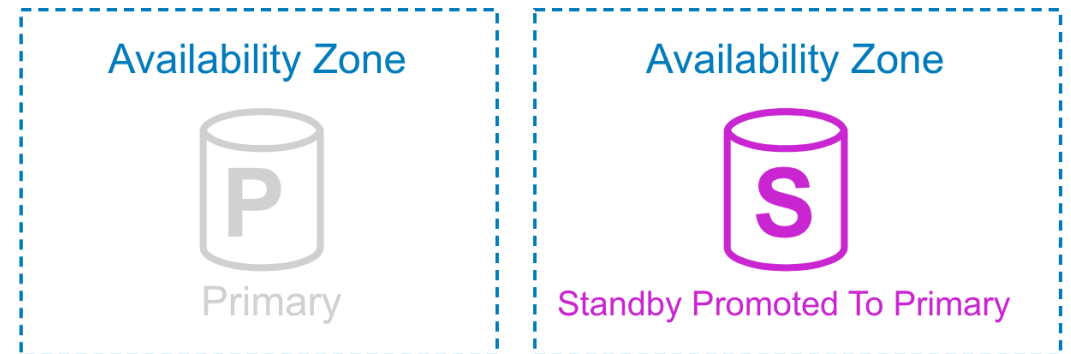
Multi-AZ is when you have **one or two standby RDS clusters or instances** in another **Availability Zone** which fail over in the case an AZ becomes unavailable.

- **Multi-AZ Instance Deployment** is Multi AZ for **RDS Instances**
- **Multi-AZ Cluster Deployments** is Multi-AZ for **Aurora Clusters**

Multi-AZ create an exact copy of your database and data in another AZ and AWS automatically **synchronizes** changes in the database over to the standby instance



Automatic Failover protection if one AZ goes down failover will occur, and the standby instance will be promoted to primary



RDS – Multi AZ

Configuring Multi-AZ on an existing RDS instance using the **AWS CLI**

```
aws rds modify-db-instance \  
  --db-instance-identifier your-db-instance-identifier \  
  --multi-az \  
  --apply-immediately
```

If you don't apply immediately the Multi-AZ will only be provision during next maintenance window

RDS - Read Replicas

Read-Replicas allows you to **run multiple read-only copies of your database.**

The improve **read contention** which in-turn improve performance and latency

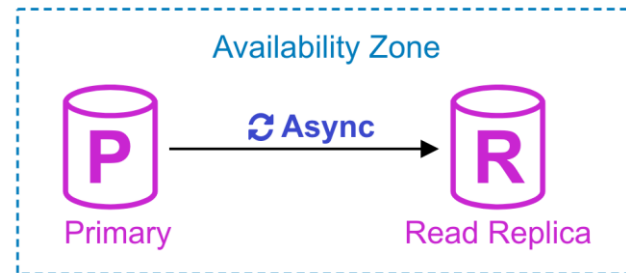
- Read contention is when multiple processes or instances competing for access to the same index or data block at the same time.



You **must have automatic backups enabled** to use Read Replicas

Asynchronous replication

occurs between the primary RDS instance and the replicas.

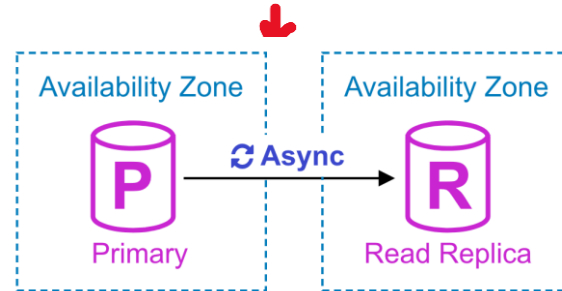


- You can have **up-to 5** replicas of a database for MySQL, MariaDB, and PostgreSQL
 - You can have up-to 15 read replicas for Aurora
- Each Read Replica will have its own DNS Endpoint
- Read Replicas will by default use the same Storage Type as the source database.
 - You can change the storage type of a Read Replica

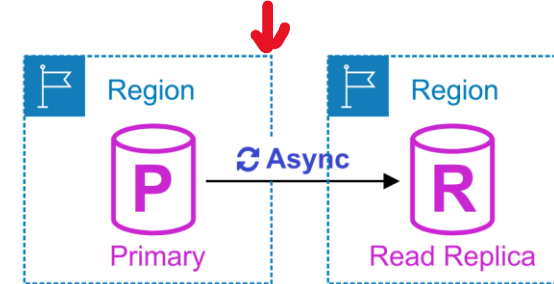
RDS - Read Replicas

You can have Multi-AZ replicas, replicas in another region, or even replicas of other read replicas

Multi-AZ Replicas



Cross-Region Replicas



Replicas can be promoted to their own database, but **this breaks replication**

No automatic failover. If the primary copy fails, you must manually update URLs to point at copy.

	MySQL and Maria DB	Oracle	Postgres	SQL Server
Replication method	Logical replication	Physical replication		
Writable?	Yes, can be enabled	No		
Can Backup?	Yes, auto and manual	Yes, auto and manual	Yes, manual	No
Parallel Replication?	Yes		No	Yes

RDS - Read Replicas

Creating a Read Replica using the **AWS CLI**



```
aws rds create-db-instance-read-replica \  
  --db-instance-identifier myreadreplica \  
  --source-db-instance-identifier mydbinstance \  
  --allocated-storage 100 \  
  --max-allocated-storage 1000 \  
  --upgrade-storage-config
```

RDS – Multi-AZ vs Read Replicas

Multi-AZ Deployments

Synchronous replication – highly durable

Only database engine on primary instance is active

Automated backups are taken from standby

Always span two Availability Zones within a single Region

Database engine version upgrades happen on primary

Automatic failover to standby when a problem is detected

Read Replicas

Asynchronous replication – highly scalable

All read replicas are accessible and can be used for read scaling

No backups configured by default

Can be within an Availability Zone, Cross-AZ, or Cross-Region

Database engine version upgrade is independent from source instance

Can be manually promoted to a standalone database instance

RDS – DB Instances

DB instance is **an isolated database environment** running in the cloud.

DB instance can **contain one or multiple user-created databases**.

Some database engines require that a database name be specified

You can have up to 40 Amazon RDS DB instances per AWS Account:

- 10 for each SQL Server edition (Enterprise, Standard, Web, and Express) under the "license-included" model
- 10 for Oracle under the "license-included" model
- 40 for Db2 under the "bring-your-own-license" (BYOL) licensing model
- 40 for MySQL, MariaDB, or PostgreSQL
- 40 for Oracle under the "bring-your-own-license" (BYOL) licensing model

Each database has user-defined **database identifier** which forms part of the DNS hostname

`https://my-rd-instance.mnopqrstuvwx.us-west-1.rds.amazonaws.com/`

and an AWS-defined RDS unique identifier



DB Identifier **identifies the RDS instance** and is not the database name.

RDS – DB Instance Classes

DB instance class just like an EC2 Instance class **determines available compute and memory or a DB instance.**

DB Instance Class types always start with db.

General-purpose

- db.m3, db.m4, db.m5, db.m6i, db.m6g, db.m7g

Memory-optimized

- Optimized X family: db.x2g, db.x2i, db.x1
- Optimized Z family (High Frequency CPU) : db.z1d
- Optimized R family (with Balanced Compute): db.r7g, db.r6g, db.r6i, db.r5b, db.r5d, db.r5, db.r4, db.r3

Burstable-performance

- **db.t2**, db.t3, db.t4g

Optimized Reads

- db.r6gd, db.r6id

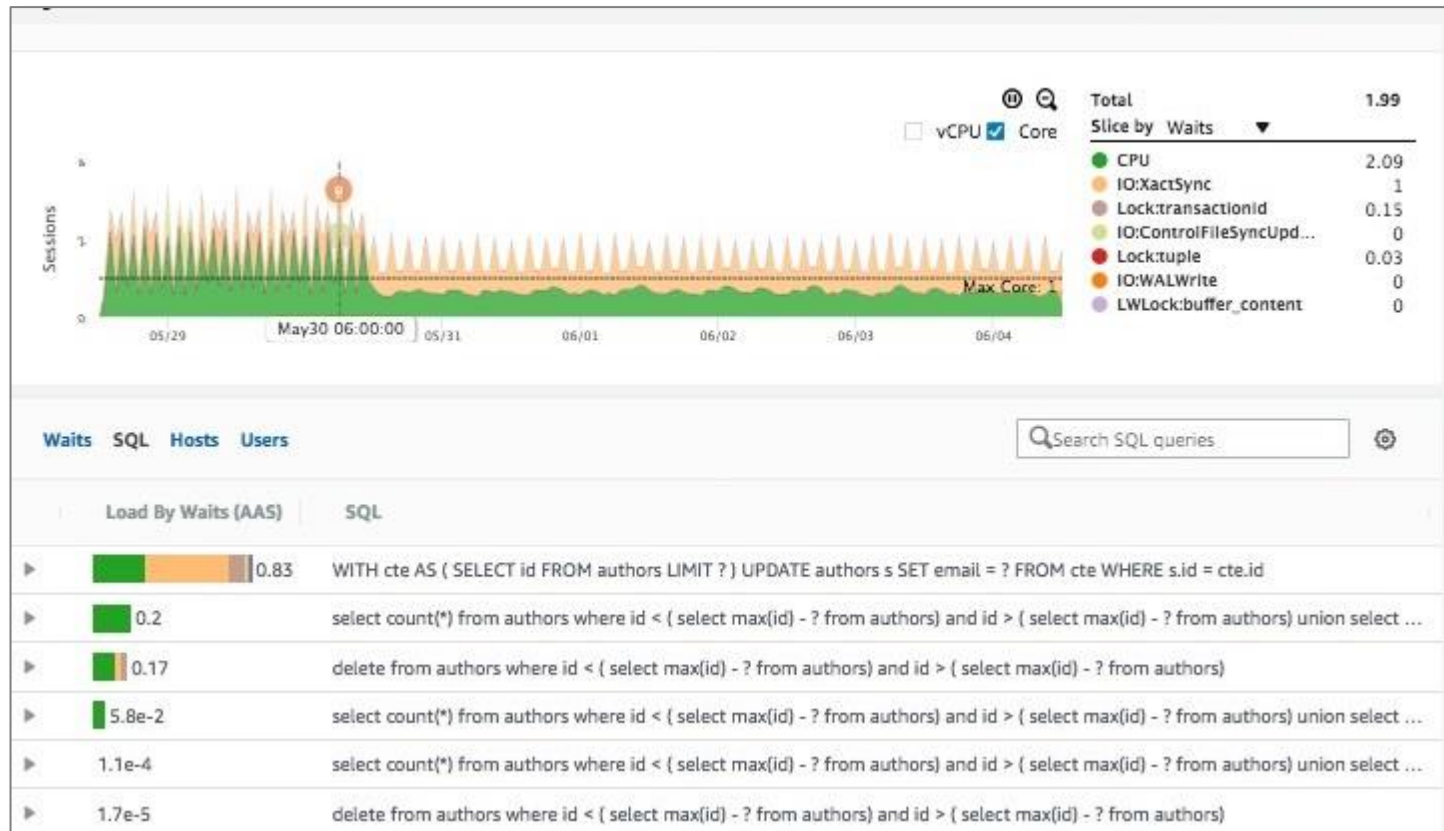
RDS – DB Instance Storage

DB instances use Elastic Block Storage **(EBS) volumes for database and log storage.**

- You can utilize
 - General Purpose SSD (gp2, gp3)
 - Provisioned IOPS SSD (io1, io2 Block Express)
 - Magnetic
 - Not recommended
- maximum storage that most DB instance classes is **64 TB**
 - Will greatly vary based on engine type and instance type and size
- You can increase storage size of an EBS
 - RDS will not let you reduce the size of storage, only increase
 - To decrease storage size create a new DB instance that has less provisioned storage space

RDS – Performance Insights

RDS Performance Insights helps you easily identify bottlenecks and performance issues. By default, Performance Insights is turned on providing 1 week of performance data



For additional cost you can change retention period to **2 years**.

RDS Custom

RDS Custom automates database administration tasks and operations
Allows customers to **directly manage aspects of RDS instead of AWS** for
companies require third party applications or customizations for their databases.

What you can do:

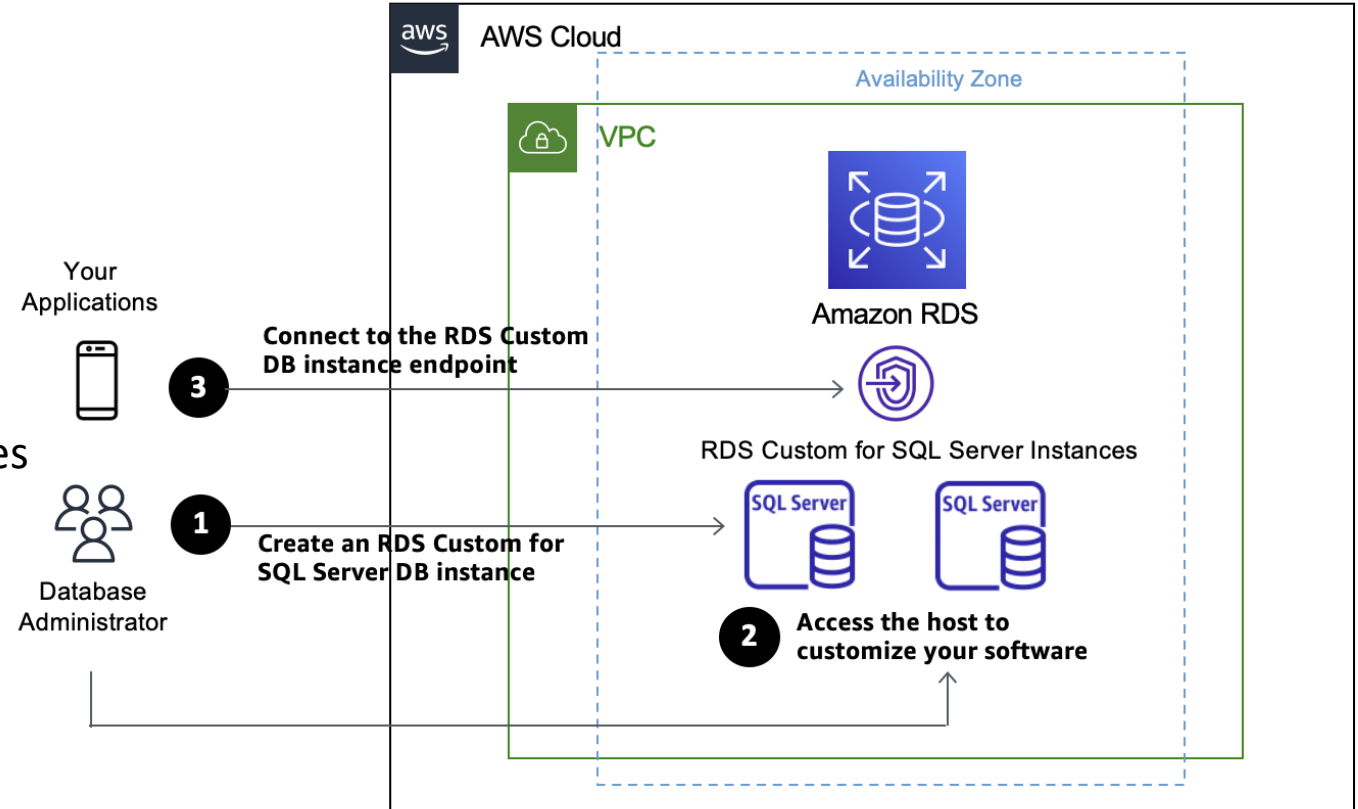
- Install third-party applications.
- Install custom patches.
- Create your own automation.

How does it work:

- You create RDS Custom DB instances
- You connect a RDS Custom DB instance endpoint
- You can directly access the host to make any changes

Works with:

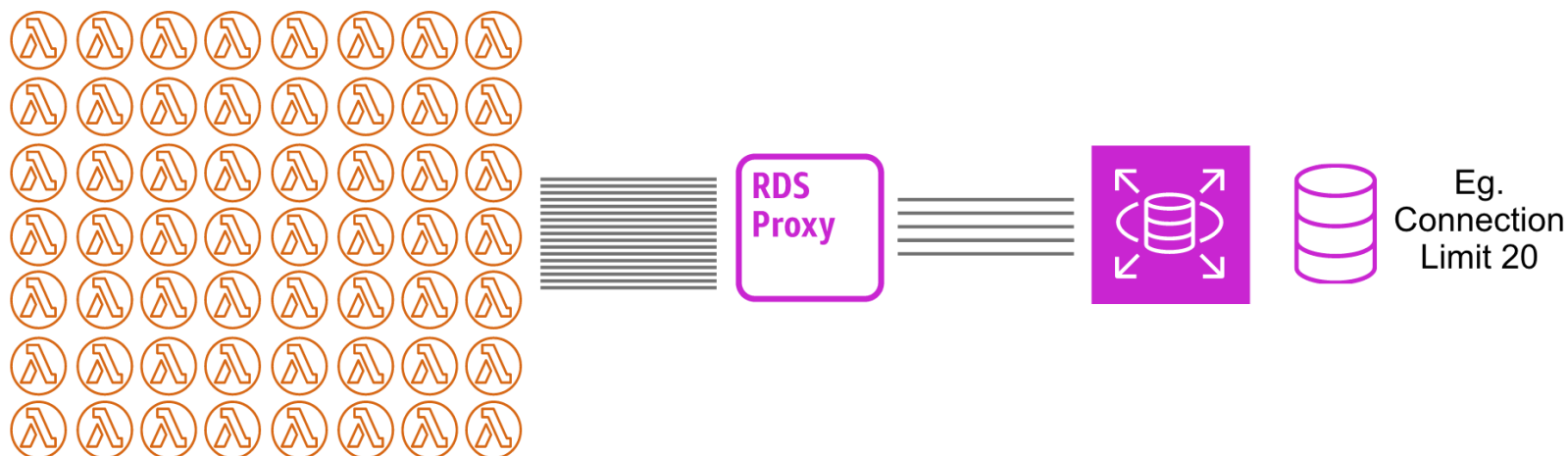
- Microsoft SQL Server
- Oracle Database



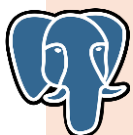
RDS Proxy



RDS Proxy create a connection pooler so that short-lived AWS Lambda functions connecting to RDS does not quickly exhaust all connections.



A connection pooler reuses existing connections reducing the amount connections opening and closing.



RDS Proxy basically does what something like PGBouncer does for Postgres. PGBouncer is an open-source connection pooler.

Optimized Reads and Writes



RDS Optimized Reads and Writes allow database operations to maximize performance, efficiency, and throughput:

- Writes 0.5x faster
- Reads 2x faster

RDS Optimized Reads and Writes **utilizes NVMe-based SSD block storage instead** of AWS EBS for temporary tables for greater performance

Queries that use temporary tables:

- sorts, hash aggregations, high-load joins, and Common Table Expressions (CTEs)

RDS Optimized Reads and Writes are available for specific combination of Instance Classes and Engine Versions: eg.

- db.r5b + MySQL 8.0
- Some DB engines only allow for optimized reads
- Reads and Writes have different requirements
- Additional database configuration may be required to take advantage of optimize reads and write

RDS – IAM Authentication

IAM Authentication allows you to **authenticate with an IAM authentication token to an RDS instance's database** instead of using a password

Works with:

- MySQL
- Maria DB
- Postgres



An **authentication token** is a unique string of characters that Amazon RDS generates on request using AWS Signature Version 4 (AWS SigV4)

- Each token has a lifetime of 15 minutes
- You can also still use standard database authentication alongside IAM Authentication
- Users can use IAM Authentication instead of having to use a password.
- EC2 instances can use IAM Authentication instead of having to use a password

Enabling IAM Authentication on an RDS instance

```
aws rds modify-db-instance \  
--db-instance-identifier mydbinstance \  
--apply-immediately \  
--enable-iam-database-authentication
```

RDS – IAM Authentication

Create a policy and attach to user or role to allow ability to authenticate as specific users.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "rds-db:connect"
      ],
      "Resource": [
        "arn:aws:rds-db:us-east-2:123456789012:dbuser:my-hello/brown",
        "arn:aws:rds-db:us-east-2:123456789012:dbuser:db-hello/bayko"
      ]
    }
  ]
}
```

Create db users and grant database access (Postgres example)

```
CREATE USER brown; GRANT db-hello TO brown;
CREATE USER bayko; GRANT db-hello TO bayko;
```

Generate Auth Token to be used in place of password when authenticating

```
export RDSHOST="db-hello.123456789012.ca-central-1.rds.amazonaws.com"
export PGPASSWORD="$(aws rds generate-db-auth-token \
--hostname $RDSHOST \
--port 5432 \
--region ca-central-1 \
--username brown )"
```

RDS –Kerberos Authentication

Kerberos is a **network authentication protocol** which is also **directly integrated into Microsoft Active Directory**

RDS support for Kerberos and Active Directory provides the benefits of single sign-on and centralized authentication of database users.

Works with:

- AWS Directory Service for Microsoft Active Directory
- Your own on-premises Active Directory.

Can be used with:

- Microsoft SQL Server
- Postgres
- MySQL
- Oracle

- Microsoft SQL Server and PostgreSQL DB instances support one and two-way forest trust relationships.
- Oracle DB instances support one-and two-way **external** and forest trust relationships.

RDS – Secrets Manager Integration

AWS Secrets Manager **can manage an RDS instance's master user password**, allowing it to be rotated out.

Does not work with:

- Microsoft SQL Server
 - Amazon RDS Blue/Green Deployments
 - Amazon RDS Custom
 - Oracle Data Guard switchover
 - RDS for Oracle with CDB
- The secret will be rotated every seven days by default
 - Web-apps need to be configured to access the password programmatically from Secrets Manager
 - If you delete a DB instance the secret is also deleted

Let Secrets manager **manage** master user password for RDS

```
aws rds modify-db-instance \  
--db-instance-identifier mydbinstance \  
--db-instance-class db.m5.large \  
--apply-immediately \  
--manage-master-user-password
```



RDS generates the master user password and manages it throughout its lifecycle in Secrets Manager.

RDS – Master User Account

Master User Account in RDS is the initial database account that's created when you provision a new DB instance.

This account is granted **full administrative privileges** on the database:

- Creating tables
- Creating schemas
- Performing SQL operations

Master User Account username and password is set at the **time of creation** of the RDS instance

```
aws rds create-db-instance \  
--db-instance-identifier mydbinstance \  
--allocated-storage 20 \  
--db-instance-class db.t3.micro \  
--engine mysql \  
--master-username mymasteruser \  
--master-user-password mysecurepassword
```



The username will be visible in the AWS console.

Its **recommended to not directly** use the Master User Account for daily use

- Instead. You should create database users with the least of amount of permission to perform specific duties

You can **reset** the Master User Account password:

```
aws rds modify-db-instance \  
--db-instance-identifier your-instance-identifier \  
--master-user-password 'new_password' \  
--apply-immediately
```



Password format and restrictions varies per database engine but generally passwords need to be at least 8 characters long.

RDS – Activity Streams

Database Activity Streams allows you to **control administrator access to data streams** to secure both external and internal security threats

Turning on Activity Streams via the **AWS CLI**



```
aws rds start-activity-stream \  
  --mode async \  
  --kms-key-id my-kms-key-arn \  
  --resource-arn my-instance-arn \  
  --engine-native-audit-fields-included \  
  --apply-immediately
```

- Amazon RDS pushes activities to an Amazon Kinesis data stream in near real time.
- Kinesis stream is created automatically.
 - activity streams feature in Amazon RDS is free Kinesis is not
- From Kinesis, you can monitor the activity stream, or other services and applications can consume the activity stream for further analysis.

RDS – Parameter Groups

Parameter group acts as a **container for engine configuration values** that are applied to one or more DB instances. Parameter groups let you change database parameters specify how the database is configured.

Modifying a parameters group to change database parameters via **AWS CLI**

```
aws rds modify-db-parameter-group \  
--db-parameter-group-name mydbparametergroup \  
--parameters "ParameterName=max_connections,ParameterValue=250,ApplyMethod=immediate" \  
              "ParameterName=max_allowed_packet,ParameterValue=1024,ApplyMethod=immediate"
```



Each database engine will completely different database parameters:

Postgres database parameter examples:

- `work_mem`: Memory for sort operations; increase for complex queries.
- `shared_buffers`: Memory for shared buffers; typically, 25%-40% of system memory.
- `maintenance_work_mem`: Memory for maintenance operations; increase for faster vacuuming/indexing.
- `effective_cache_size`: Helps query planner with memory available for caching.
- `checkpoint_completion_target`: Spreads checkpoint writes; closer to 1.0 for even spread.
- `wal_buffers`: Size of WAL buffer; increase to batch writes and reduce I/O.

RDS – Public Accessibility

RDS Public Access(sible) option changes if the DNS Endpoint will resolve to the private IP address from **traffic from outside the VPC**.



```
aws rds create-db-instance \  
--db-instance-identifier mypublicdbinstance \  
--db-instance-class db.t3.micro \  
--engine mysql \  
--allocated-storage 20 \  
--master-username adminuser \  
--master-user-password securepassword123 \  
--publicly-accessible \  
--backup-retention-period 7 \  
--engine-version 8.0.23
```

Public Access does not override Security Groups rules so ensure you allow inbound traffic on your specific DB ports.

Public Access feature is useful when you are confident with password authentication and security groups, and you want the convenience to connect to RDS instance without having to use an intermediate way of accessing the RDS instance's database.

RDS – Establishing Public Connections

Public Access RDS Connection Options

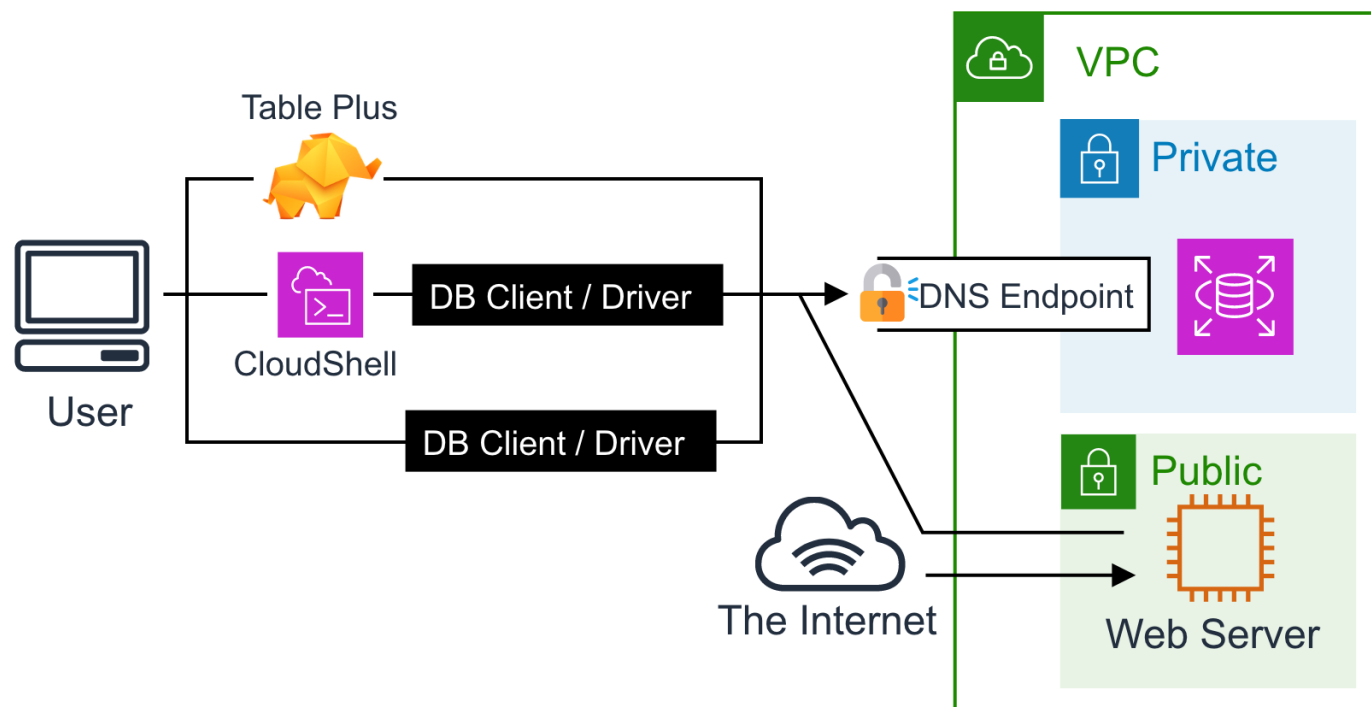
When the DNS Endpoint with Public access on there are a few options for connecting.

Use a Database management / **DB IDE** tool to establish a connection eg. TablePlus, Dbeaver, DataGrip, Navicat

Use **AWS CloudShell** and use a database client or database driver via code to establish a connection

Use a database client via your local terminal eg. psql, mysql

- Programmatically connect with a **database driver** from your language of choice eg. JDBC
 - Generally, you will do this via a web server



RDS – Establishing Public Connections

What is a connection url string?

A connection url string is a single string containing all the parameters to connect to a database.

Its a convenient way to quickly configure a connection for **database drivers and database command line clients**

The connection string may vary between database drivers and database command line clients.

- **MySQL Format:** `mysql://[hostname]:[port]/[databaseName]?[properties]`
- **MariaDB Format:** `mariadb://[hostname]:[port]/[databaseName]?[properties]`
- **PSQL Format:** `postgresql://[username]:[password]@[hostname]:[port]/[databaseName]?[properties]`
- **Oracle Format:** `oracle:thin:@[hostname]:[port]:[SID]` or `jdbc:oracle:thin:@//[hostname]:[port]/[serviceName]`
- **SQL Server Format:** `sqlserver://[hostname]:[port];databaseName=[databaseName];user=[user];password=[password]`
- `abaseName=mydatabase;user=myusername;password=mypassword`

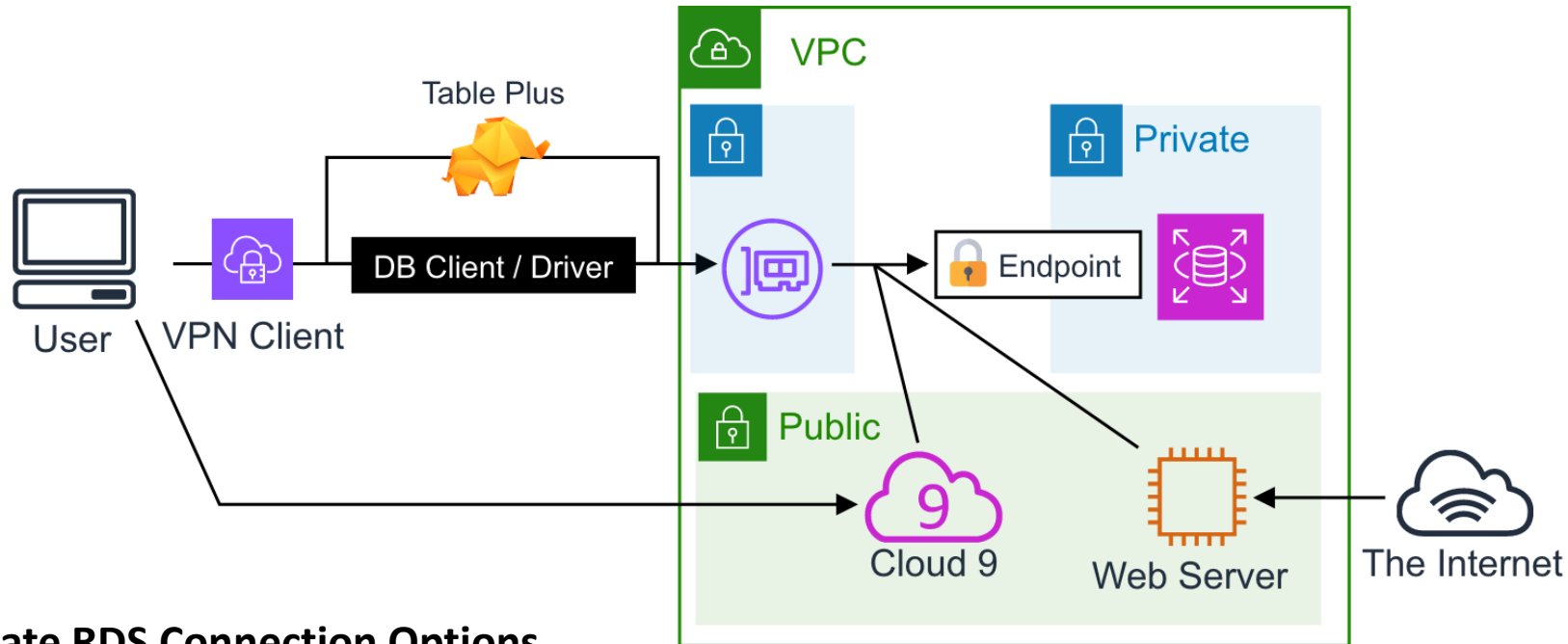
```
psql postgresql://andrew:testing123@my-db.123456789012.ca-  
central-1.rds.amazonaws.com:5432/mydatabase
```

Example of using a connection string to connect via the **PSQL command line client**

Default Ports for DB Engines

- **MySQL:** 3306
- **MariaDB:** 3306
- **PostgreSQL:** 5432
- **Oracle:** 1521
- **SQL Server:** 1433
- **Aurora MySQL:** 3306
- **Aurora PostgreSQL:** 5432

RDS – Establishing Private Connections

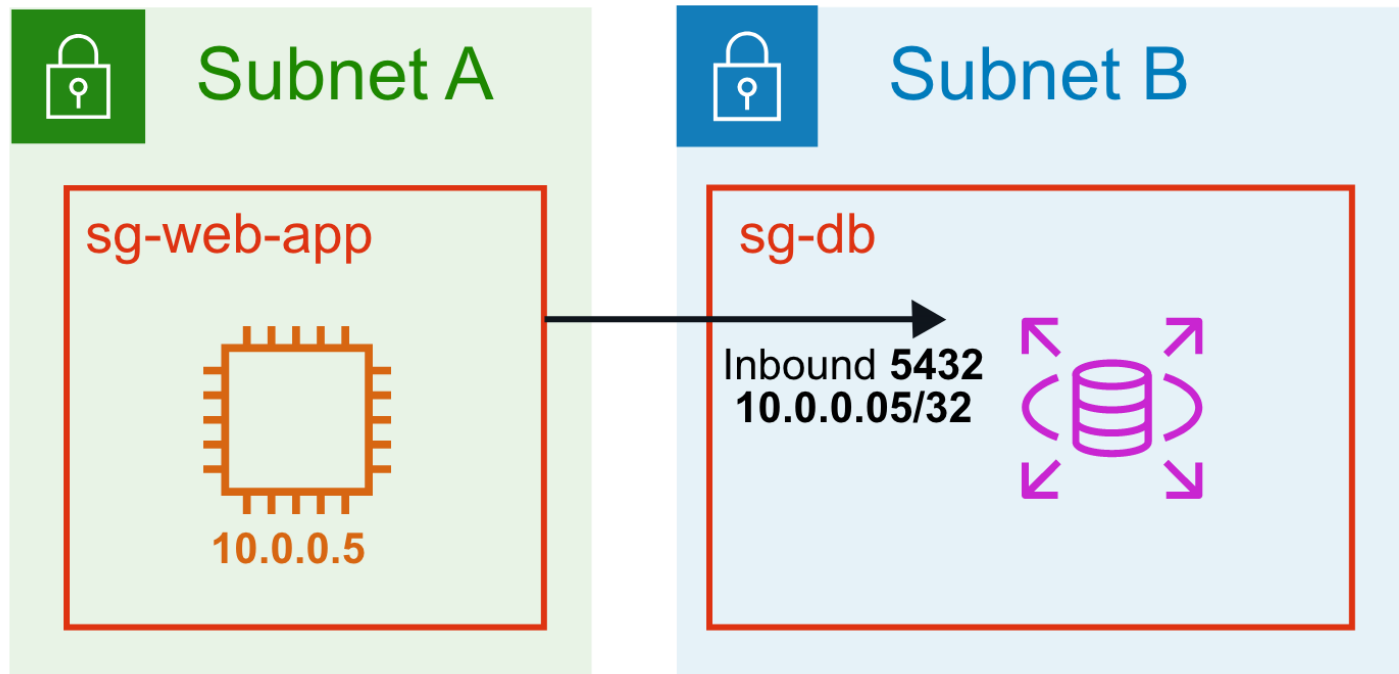


Private RDS Connection Options

- Launch a Cloud9 server (in a public subnet) in the same VPC
- Connect through a Bastion or Jumpbox and tunnel through the box
- Launch an EC2 instance and connect via SSH or Sessions manager and establish a connection
- Use AWS Client VPN to connect your machine to your VPC and establish a connection to your VPC
- For On-premise using AWS Direct Connect they can join from their on-premise network.
- AWS CloudShell can't be used because it doesn't reside within a customer manager VPC

RDS – Security Groups

An RDS Instance have a security group. **In order to establish a connection** for both public and private connections you need to open the DB ports inbound.



Default Ports for DB Engines

- **MySQL:** 3306
- **MariaDB:** 3306
- **PostgreSQL:** 5432
- **Oracle:** 1521
- **SQL Server:** 1433
- **Aurora MySQL:** 3306
- **Aurora PostgreSQL:** 5432



When a database hangs during a connection its often due to misconfigured Security Group rules.

RDS Blue Green Deployments

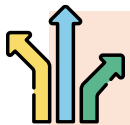


RDS Blue/Green Deployments **copies a production database environment** in a separate, synchronized **staging environment**.

- Test database changes in a safe staging environment without affecting the production environment.
- Stay current with database patches and system updates.
- Implement and test newer database features.

Creating a blue/green deployment for RDS via **AWS CLI**

```
aws rds create-blue-green-deployment \  
--blue-green-deployment-name my-blue-green-deployment \  
--source arn:aws:rds:us-east-2:123456789012:db:mydb1 \  
--target-engine-version 8.0.31 \  
--target-db-parameter-group-name mydbparametergroup \  
--target-db-instance-class db.m5.8xlarge \  
--upgrade-target-storage-config
```



Different database engines will require different prerequisites steps before replication.

RDS Extended Support

Amazon RDS Extended Support allows you to **run your database on a major engine version past the RDS** end of standard support date **for an additional cost.**

- gives you more time to upgrade to a supported major engine version
- Amazon RDS will supply patches for Critical and High CVEs as defined by the National Vulnerability Database (NVD) CVSS severity ratings
- available for up to 3 years past the RDS end of standard support date for a major engine version
 - After this point AWS will automatically upgrade your RDS engine version